# Understanding Host Network Stack Overheads

**Paper Study and Our Single Flow Experiment**

Gurudayal Meena (23110125)  Kunal Singh Rathore (23110181)
CS 331 - Computer Networks | Project 2

## I. INTRODUCTION

We studied the paper "Understanding Host Network Stack Overheads". It asks why a server can become slow at handling network traffic even when its network connection is very fast. The paper shows that the CPU and the Linux network stack can become the limiting factor.

After reading the introduction and the Linux sender/receiver data paths, we selected the Single Flow experiment from Section 3.1. We ran a smaller version on our laptop and measured TCP throughput and CPU use.

## II. PROBLEM STATEMENT

Our question was simple: how fast can one TCP flow transfer data between two programs in an Ubuntu virtual machine, and how much CPU does this use?

Input: one long-running TCP data stream generated by iPerf3. Output: received throughput and the sender/receiver process CPU readings. We used one laptop, so both programs communicated through 127.0.0.1 (loopback).

Scope: this tests local Linux TCP processing. It does not measure a real Ethernet or Wi-Fi connection. We did not reproduce NIC hardware optimizations or kernel-function CPU profiling from the paper.

## III. PROJECT OBJECTIVES

1. Understand how Linux sends and receives TCP data.

2. Run a Single Flow workload using iPerf3.

3. Record the throughput and CPU results and make graphs.

4. Explain how and why our setup differs from the research setup.

## IV. WHAT WE LEARNED FROM THE PAPER

Section 1: fast network links do not always mean fast application transfers. The CPU must still process packets and move data in memory.

Section 2: on the sender side, data passes from the application to a socket, through TCP/IP and the network-device path. On the receiver side, Linux processes arriving data and gives it to the reading application. These steps use CPU time.

Section 3.1: the researchers used one TCP flow across two physical servers connected by 100 Gbps Ethernet. They changed NIC optimizations and measured throughput per CPU core and detailed processing overhead. We studied these ideas, but our available setup allowed only the basic Single Flow workload.

## V. ARCHITECTURE AND DESIGN

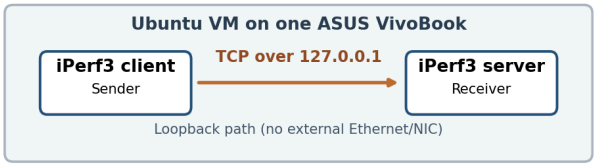



*Fig. 1. Our sender and receiver were inside the same Ubuntu VM.*

The iPerf3 client was the sender and the iPerf3 server was the receiver. Both ran inside Ubuntu in VirtualBox on an ASUS VivoBook. We used 127.0.0.1, the loopback address, so TCP data stayed inside Linux instead of going through a physical network card.

We chose this design because the paper's two dedicated servers and 100 Gbps NICs were not available to us. It let us run the TCP workload and collect basic measurements, but it cannot reproduce real NIC effects.

## VI. IMPLEMENTATION

### What we did

First, we started the receiver in one Ubuntu terminal. Then we started an iPerf3 client in another terminal. We used one TCP stream with the -P 1 option. Each run lasted 20 seconds. We repeated the client test 50 times using a Python script, with a short pause between runs.

### Commands used

```
Server: iperf3 -s -B 127.0.0.1
Client: iperf3 -c 127.0.0.1 -P 1 -t 20
```

For automated runs, our Python script called iPerf3 with -J to get JSON output. It saved the detailed result of every run as run_1.json through run_50.json. It then made single_flow_results.csv with the run number, throughput in Gbps, sender CPU, and receiver CPU.

We used Python to read these saved files and draw the graphs. During testing, we also checked active iPerf3 CPU usage using pidstat. We did not need a second laptop or change the VirtualBox network mode.

### Files we kept

single_flow_legacy.py (the original 50-run automation), results/run_1.json to run_50.json, results/single_flow_results.csv, and the graph scripts and output PNG files. The reviewed project also has

validate_data.py and analyze_results.py for checking and plotting the saved results.

## VII. TESTBED SETUP

| Item | Our setup |
|---|---|
| Computer | ASUS VivoBook (one laptop) |
| System | Ubuntu 20.04-based VirtualBox VM |
| Linux kernel | 5.15.0-139-generic |
| Guest CPU / RAM | 4 virtual CPUs / about 3.8 GiB |
| Benchmark | iPerf3 3.7, TCP, one stream |
| Connection | 127.0.0.1 (loopback) |
| Run settings | 20 seconds; 50 runs; 2-second pause |

The exact physical CPU model was not recorded. Since both programs ran inside one VM, they shared its available CPU and memory resources.

## VIII. EVALUATION

### Throughput results from 50 runs

| Measurement | Recorded result |
|---|---|
| Average throughput | 55.19 Gbps |
| Minimum | 39.44 Gbps |
| Maximum | 60.07 Gbps |
| Standard deviation | 3.30 Gbps |

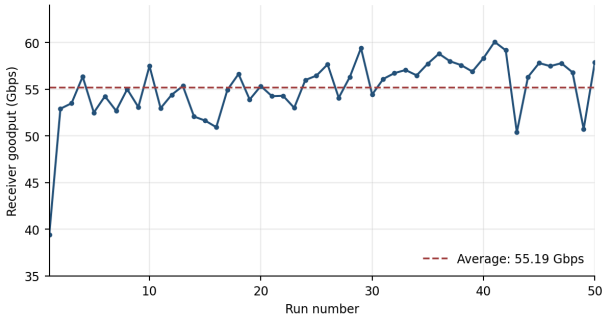



*Fig. 2. Received TCP goodput in each of our 50 recorded runs.*

The average of the 50 runs was 55.19 Gbps. Some runs were faster than others. The first run was the slowest, at 39.44 Gbps. We kept it in our results instead of removing it.

### CPU results

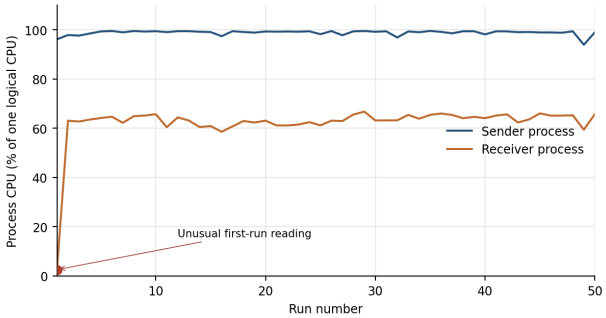



*Fig. 3. iPerf3 process CPU readings for sender and receiver.*

The sender process used about 98.84% of one logical CPU on average. The receiver usually used around 60-66% of one logical CPU. The first run gave an unusually low receiver reading (2.58%), so this point should not be treated as a typical receiver result. Most measured process CPU time was in system mode.

### Why our 55 Gbps is not a real NIC result

Both programs talked over the loopback interface. No data crossed the laptop's physical Ethernet or Wi-Fi interface. The number shows local TCP goodput inside the Ubuntu VM, not the bandwidth of the laptop's network card.

The paper reports about 42 Gbps per bottleneck CPU core with suitable NIC optimizations. That is a different measurement in a different setup. We cannot say our laptop beat the paper's servers. Also, our CPU numbers are iPerf3 process CPU use; they are not a breakdown of individual kernel functions.

The change between runs may be affected by host load, CPU scheduling, or VM activity, but we did not run controlled tests to prove one exact cause.

## IX. SUMMARY

We learned the basic sender and receiver paths of the Linux TCP stack and ran the Single Flow workload using iPerf3 inside one Ubuntu VM. We collected 50 real measurements, saved the JSON and CSV data, and made throughput and CPU graphs. The average local TCP goodput was 55.19 Gbps.

Our work covers the single-flow workload only. We did not reproduce the paper's physical 100 Gbps testbed, TSO/GRO and aRFS comparison, DDIO/cache experiments, or detailed CPU-cycle breakdown. In future, a two-machine physical-network test and controlled CPU profiling would give a closer comparison.

## X. REPRODUCIBILITY

Project_GitHub_link:https://github.com/kunalsinghrathore23110181/CN_Project_2.git

From the single_flow_experiment directory, use two terminals:

```
Terminal 1: iperf3 -s -B 127.0.0.1
Terminal 2: iperf3 -c 127.0.0.1 -P 1 -t 20
```

To analyze the 50 existing runs without changing the data:

```
python3 validate_data.py
python3 analyze_results.py --input results
```

To collect new runs with the reviewed script (in a new results folder):

```
python3 run_vm.py --runs 50 --duration 20 --pause 2
```

The saved 50-run results in this report come from the original Python automation, not from a new run of run_vm.py. Raw data is under single_flow_experiment/results/.

## REFERENCES

[1] Q. Cai, S. Chaudhary, M. Vuppalapati, J. Hwang, and R. Agarwal, "Understanding Host Network Stack Overheads," ACM SIGCOMM, 2021. doi:10.1145/3452296.3472888.

[2] Research code: https://github.com/Terabit-Ethernet/Understanding-network-stack-overheads-SIGCOMM-2021

## APPENDIX A: TEAM WORK DISTRIBUTION AND MEMBER RATING

A.1 Work distribution

Fill in the actual work completed by each team member after discussing it together. As the template requires, the two team ratings must be different integers from 1 to 10. No ratings or individual task claims have been guessed here.

| # | Member name | Roll no. | Tasks done | Rating (unique) |
|---|---|---|---|---|
| 1 | Gurudayal Meena | [23110125] | Studied the research paper, worked on Python automation and data analysis, generated throughput and CPU usage graphs, and prepared the report and presentation. | [10] |
| 2 | Kunal Singh Rathore | [23110181] | Studied the research paper, set up the Ubuntu VirtualBox environment, ran the iPerf3 sender and receiver, performed the Single Flow experiment, and collected the experimental data. | [9] |